PEAK SURFER: A Mode for Frequency Hopping

Jamie Jorgensen
Mark Motley

National Security Agency
9800 Savage Road, Fort Meade, MD 20755, USA

{jbjorge, mjmotle}@nsa.gov

Abstract

We describe a simple mode that makes use of a wide block cipher (128 bits wide for example) along with a narrow lightweight block cipher (32 bits wide for example) to construct a frequency hopping algorithm.

1. Introduction

As cryptographers we tend to think in terms of COMSEC, i.e. protecting the confidentiality or integrity of a transmitted message. However, it is also important to provide TRANSEC – to ensure availability, low probability of detection, etc. for the transmission, independent of the particular information that transmission contains.

There are many different TRANSEC services, and many different ways of providing these services. Here we will focus on *availability*. That is, we aim to guarantee that an adversary is unable to interfere with a transmission in such a way as to prevent the intended recipients from receiving it.

Among the various methods of providing availability, we focus here on frequency hopping. We propose a simple mode for constructing a frequency hopper from common symmetric components and analyze its security.

2. Frequency Hopping

We begin by establishing an abstract model for what a frequency hopper does. Suppose we have users $1, \ldots, n$, and user i is allocated a channel of bandwidth w_i. The channels selected all lie in the frequency range $[0, W]$ and

$$\sum_{i=1}^{n} w_i = W - s,$$

where s is the *slack* (i.e. the unused part of the frequency range). The channels assigned to different users can only overlap at their endpoints, since otherwise their communications will interfere with one another.

At each time step t we want to randomly permute the intervals, and *randomly* position them using the available slack (see Figure 2.1 for an illustration).

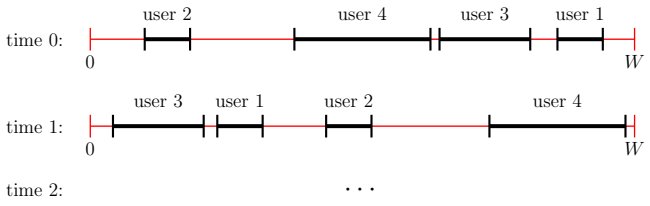

Figure 2.1: An Illustration of Frequency Hopping

We can formalize this to get the following definition:

Definition. A *frequency hopping algorithm* is a function $H\colon K \times \mathbb{Z}_{\geq 0} \to [0, s]^n$, along with a method for ordering coordinates of $H(k, t)$ that are equal.

K here is the cryptovariable space, and $\mathbb{Z}_{\geq 0}$ signifies the times t at which new assignments (i.e., hops) are made; nominally times $t = 0, 1, 2, 3, \ldots$.

This is what an assignment of intervals to users would look like if each width w_i were 0. But from the value of $H(k, t)$ and the width data $w_1, w_2, \ldots, w_n$, we can "blow up" points to intervals of the appropriate widths, without introducing overlaps (except perhaps at endpoints). Suppose $H(k, t) = (a_1, a_2, \ldots, a_n) \in [0, s]^n$. It suffices to assign *left endpoints* to the users. We do this as follows:

1. Sort the values $a_1, a_2, \ldots, a_n$ to get $a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}$ with ties broken in some predetermined way, say in favor of the user with the lower index.

2. Assign user $\pi(1)$ the left endpoint $a_{\pi(1)}$.

3. For $i = 2, 3, \ldots, n$, assign user $\pi(i)$ the left endpoint

$$a_{\pi(i)} + (w_{\pi(1)} + w_{\pi(2)} + \cdots + w_{\pi(i-1)}).$$

Figure 2.2: An Abstract Frequency Hopper

3. A Simple Frequency Hopping Mode

We can now propose a simple mode for frequency hopping. In order to instantiate this mode we require a 128-bit block cipher $\mathcal{E}_k$ with 256-bit cryptovariable as well as a 32-bit block cipher E_k with 128-bit cryptovariable.

We can then define a frequency hopper $H\colon \mathbb{Z}_{\geq 0} \times V^{256} \to [0, s]^n$ by

$$H(k, t) = \left(\frac{E_{k_t}(0) \cdot s}{2^{32}}, \frac{E_{k_t}(1) \cdot s}{2^{32}}, \ldots, \frac{E_{k_t}(n-1) \cdot s}{2^{32}} \right),$$

where s is the slack. At time t the users perform the operations shown in Figure 3.1 in order to determine the frequency ranges assigned to the various users (see Figure 3.2 for a schematic).

For each hop, we have to perform one encryption using $\mathcal{E}$ and n encryptions using E. Note that, since E_k is a permutation, there is no need to define a tie breaking mechanism for this scheme.

Note also that here `IV` represents an input that, although not necessarily secret, is a distinct series of values included to prevent repeats in the sequence of keys. Since the goal here is to prevent jamming, the users cannot communicate to agree on a value of `IV`, which could instead be set to, for example, the current day. The number of bits reserved for the counter must be sufficient for the chosen hop rate and cryptoperiod (for example a 45-bit counter is sufficient for 2^{20} hops per second and a one year cryptoperiod).

1. Compute $k_t = \mathcal{E}_{\mathtt{CV}}(\mathtt{IV} \,||\, t)$.
2. Compute $H(t) = (a_i^t) = (E_{k_t}(0)s/2^{32},\, E_{k_t}(1)s/2^{32}, \ldots, E_{k_t}(n-1)s/2^{32})$.
3. Sort the list to get $a^t_{\sigma(1)} < \cdots < a^t_{\sigma(n)}$.
4. Set $L^t_{\sigma(i)} = a^t_{\sigma(i)} + \sum_{j=1}^{i-1} w_{\sigma(j)}$ for $1 \le i \le n$.
5. Set $C^t_{\sigma(i)} = [L^t_{\sigma(i)}, L^t_{\sigma(i)} + w_{\sigma(i)}]$.

Figure 3.1: A Frequency Hopping Mode

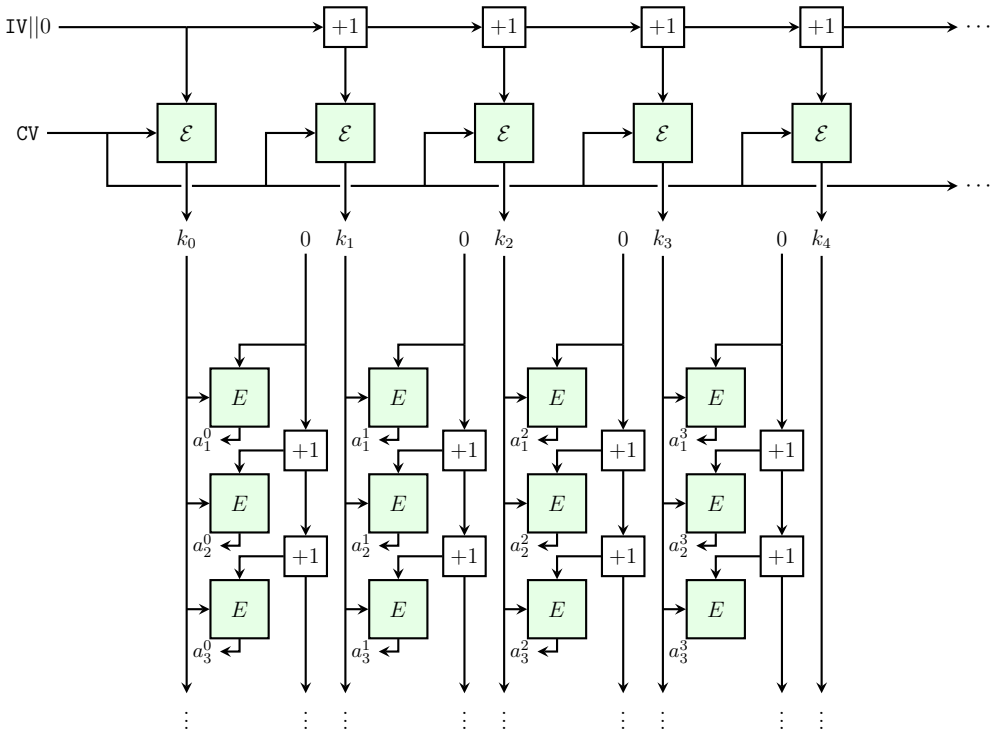

Figure 3.2: Frequency Hopper Mode Schematic

$\mathcal{E}$ must be a fully secure block cipher, but this is not the case for the narrow block cipher. The adversary does not have the ability to spoof E – at each hop all the data that is available is $\{E_k(0), \ldots, E_k(n-1)\}$. In particular, the

amount of data available is limited by the number of users in the system. A conservative upper bound is 2^{16} users, compared to the 2^{32} data that would otherwise be available.

An adversary that is able to observe the frequency bands that have been assigned to some subset of T of the users effectively has a collection of T matched plain/cipher pairs for E_{k_t}. If the adversary is able to attack E with this data, they will recover k_t, and so be able to deduce channels assigned to the remaining users. Since k_t is an output of $\mathcal{E}$ for a known input, it also recovers a matched plain/cipher pair for this block cipher.

Since $\mathcal{E}$ is assumed to be a secure block cipher, this doesn't benefit the attacker. Since our goal is to thwart a jammer, learning the channel location of a user after the end of the current hop is also of no value, as the location of that user's channel during the next hop will depend on a new key. So, in addition to limiting the data available to the attacker, we only have to defend against attacks that can be completed within a dwell, i.e. the duration of a hop. The dwell time has to be kept small at any rate to defeat so called "fast follower" jammers, and so we're targeting a rate in excess of 10000 hops per second. This leaves the adversary with a very limited time in which to complete its attack.

4. Caju, a 32-bit Lightweight Block Cipher

Our mode requires both a 128-bit block cipher and a 32-bit block cipher. Many secure 128-bit block ciphers already exist, so that there is no compelling reason to propose a new one for this purpose. However, 32-bit block ciphers are less common. Also, this particular application is somewhat unusual in its requirements, so that a custom design for the 32-bit block cipher component makes sense.

Caju is a 32-bit substitution-permutation network (SPN) block cipher. The encryption function consists of twelve cryptovariable-dependent rounds each composed of an s-box layer, followed by a linear map, followed by the `XOR` of a 32-bit round key.

The Caju round function acts on a state consisting of four 8-bit words (w, x, y, z). In this representation, the s-box layer consists of 8 identical 4-bit s-boxes, with the i-th s-box acting on the i-th bit level of the state, i.e. (w_i, x_i, y_i, z_i). The s-box is a composition of four Toffoli gates (3-bit to 3-bit gates of the form $(a, b, c) \mapsto (a, b, c + ab)$). See Figure 4.1 for a diagram.

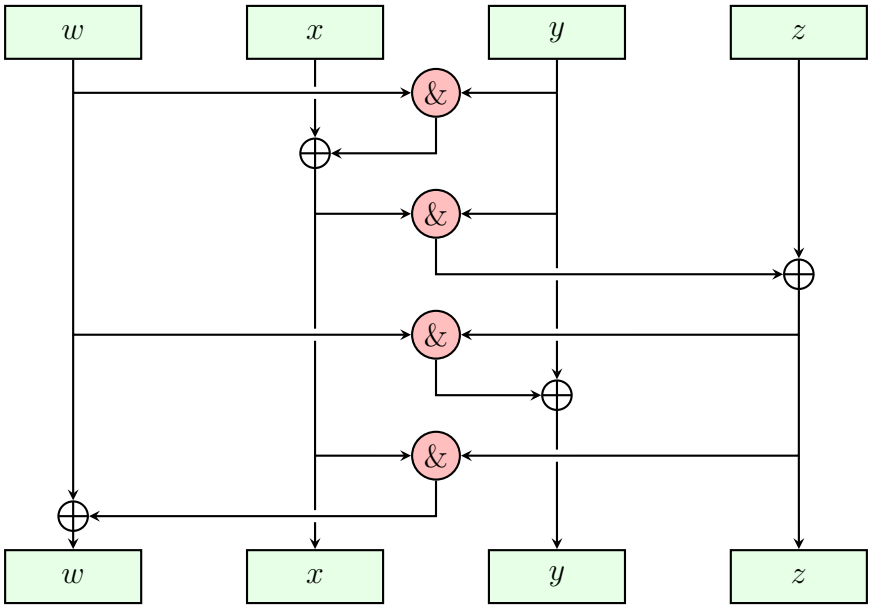

Figure 4.1: Caju s-box Layer

The linear map for Caju mixes the w and x words using `XOR`s and circular shifts, leaving the y and z words unaffected. Then, the left and right two words are swapped. See Figure 4.2 for a diagram.

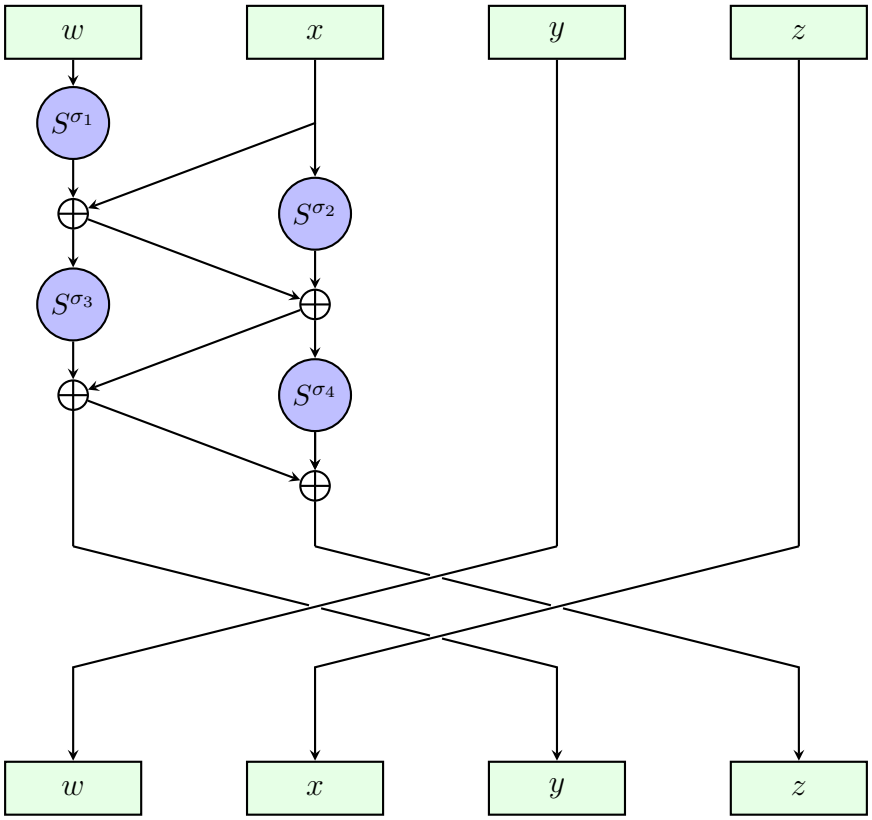

Figure 4.2: CAJU Linear Map

The parameters for the CAJU linear map are $\sigma_1 = 2, \sigma_2 = 6, \sigma_3 = 1, \sigma_4 = 7$. If we denote the s-box layer by π and the linear layer by L, then one round of CAJU is the composition $x \mapsto \tau_{k_i}(L(\pi(x)))$, where τ_{k_i} is the `XOR` of the i-th round key. There is a pre-add of key before the first round, and the total number of rounds is 12.

The total number of round keys required is 13. The block cipher has a simple, linear key schedule. If the four 32-bit words of cryptovariable are $\texttt{CV}_0, \ldots, \texttt{CV}_3$, then we initialize the key registers with $k_i = \texttt{CV}_i$ for $0 \leq i \leq 3$. The i-th round key is computed as $k_0 \oplus i$, and the key register is updated by the rule

$$(k_0, k_1, k_2, k_3) \mapsto (k_1, k_2, S^{19}(k_0 \oplus k_1) \oplus k_3, k_0 \oplus k_1)$$

(see also Figure 4.3).

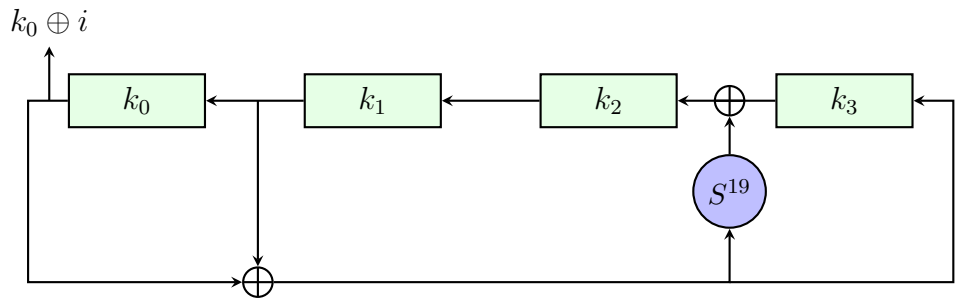

Figure 4.3: CAJU Key Schedule

5. Adversary Model

In order to evaluate the security of our proposed frequency hopper, we need a model of the attacker and some figure of merit for how well our algorithm thwarts its efforts to interrupt our communications.

There is a huge space of possible jamming strategies available to the adversary: It can broadcast any signal it likes within the physical limitations imposed by its available resources. In the interest of designing a model simple enough that we can understand it, we restrict to *point jammers*. In other words, we assume the jammer selects a frequency (possibly changing over time), and transmits a signal at this frequency. Our working assumption is that whenever the selected frequency falls within the band assigned to some channel, that channel is jammed. Although this model is undoubtedly an oversimplification, our hope is that it is accurate enough that a design that performs well in this model also performs well against real-world adversaries.

In evaluating the performance of a frequency hopper we focus on a single channel. In other words, for a fixed channel, how often is the jammer successful in jamming this channel? If this channel occupies a proportion p of the total available bandwidth, then clearly the jammer can succeed with probability p by selecting a frequency uniformly at each dwell. In general the jammer will be able to do better than this, independent of the frequency hopping algorithm. For example, in the case of a single channel hopping within a band less twice the width of the channel, the jammer succeeds with probability 1 by choosing to jam the center frequency at each dwell.

This suggests that a reasonable measure of success for a frequency hopper is that the maximum probability that a frequency is covered by a fixed channel during the hopping process is as small as possible. Note that for our proposal, when all the channels are of the same width, all the channels' positions have the same distribution.

For the PEAK SURFER frequency hopper, this maximum probability can be easily computed as a function of the relevant parameters (size and number of the channels, size of the slack). We omit the details.

6. Performance

Bitsliced implementations are well suited for algorithms that can be computed as a simple circuit using `XOR`s, `AND`s, and `OR`s. In a bitsliced implementation, each register is dedicated to storing one bit of multiple inputs rather than a single complete input.

Define the following notation:

Variable Name	Value	Description
REGISTER_SIZE	32 or 64, usually	The size of the registers being used.
USERS	REGISTER_SIZE × SSIZE	The number of users that will be hopped via the frequency hopper. This should be a multiple of REGISTER_SIZE.
SSIZE	USERS/REGISTER_SIZE	Number of registers in width of state space; Used as a looping variable end condition
USER_ID	$\in [0, \text{USERS})$	The particular user's index in the state space.
USER_INDEX	$\in [0, \text{REGISTER\_SIZE})$	The particular user's index in a register.
USER_MASK	$\texttt{0x1} \ll (\text{USER\_ID}\%\text{REGISTER\_SIZE})$	A mask used to pick off the bits of a user's index.
S		State array for bitsliced computation
$S[i]$	32 REGISTER_SIZE registers	A subspace of the state S containing the state for a block of REGISTER_SIZE users
$W.bit[i]$	REGISTER_SIZE-bit value	i^{th} bit-level of a word W
key	32-bit value	round key

Table 6.1: Glossary of variables

The Caju algorithm can be implemented using a bitsliced implementation in order to improve performance. USER_IDs are initialized to values of a one-up counter. So for J users, user j will be given a USER_ID initialized to j, where $j \in [0, J-1]$. In order to accommodate values of J larger than REGISTER_SIZE, multiple registers will be used for each bit level. It should be noted that it is more efficient to compute additional dummy encryptions rather than to try to zero out the unused USER_IDs. The highest efficiency is achieved when the number of users is a multiple of REGISTER_SIZE. Details of the Caju bitsliced implementation can be found in the appendix.

Bit Level 31	$X_{31_{J-1}}$	$X_{31_{J-2}}$	...	X_{31_1}	X_{31_0}
Bit Level 30	$X_{30_{J-1}}$	$X_{30_{J-2}}$	...	X_{30_1}	X_{30_0}
$\vdots$	$\vdots$	$\vdots$		$\vdots$	$\vdots$
Bit Level 1	$X_{1_{J-1}}$	$X_{1_{J-2}}$	...	X_{1_1}	X_{1_0}
Bit Level 0	$X_{0_{J-1}}$	$X_{0_{J-2}}$	...	X_{0_1}	X_{0_0}

Table 6.2: Visual representation of a bitsliced state. The j^{th} column corresponds to USER_j.

At each hop, all USER_IDs are encrypted and USER_j needs to compute its index. In other words, USER_j needs to know how many users are to its left (how many users have smaller encrypted values than USER_j). Note that each user *does not* need to know a complete ordering of all users, but rather just a count of how many users have encrypted values smaller than their own. Algorithm 6.1 will compute a user's index by maintaining separate counts of how many values are larger and smaller than USER_j. By looking at each bit level b of the state, USER_j will learn that some user indices are larger or smaller than theirs based on whether USER_j's bit value at that level is 0 or 1 respectively; only one mask is updated during the evaluation of each bit level.

Caju, including the SPECK key schedule (one possible way to generate 128 bits of key for Caju) and index computation, was implemented in C. Table 6.3 displays performance numbers for an Intel Xeon E5-2660v4 2.00GHz CPU with gcc version 9.2.0.

Number of Users	32	64	128	256
32-bit registers	1,865,000	999,000	628,000	354,000
64-bit registers	-	1,801,000	**877,000**	503,000

Table 6.3: Hops per Second Performance for PEAK SURFER

Algorithm 6.1 USER_j Index Computation

$X[\,] = E_k(\text{USER}_{J-1}...\text{USER}_0)$ `//bitsliced CAJU encryption`
$t = E_k(\text{USER}_j)$ `//CAJU encryption of j`
for b=31 **to** 0 **do**
  $T[b] = -1 * t_b$
end for
MASK_B = `0x0 //marks users bigger than j`
MASK_S = `0x0 //marks users smaller than j`
for b=31 **to** 0 **do**
  $\text{MASK}_B \,|= (X[b] \,\&\, (\sim(\text{MASK}_S \,|\, T[b])))$ `//update MASK_B`
  $\text{MASK}_S \,|= ((\sim(X[b] \,|\, \text{MASK}_B)) \,\&\, T[b])$ `//update MASK_S`
end for
return `popcount`(MASK_S) `//set bits are users less than j`

Table 6.3 was computed using the -O3 flag in the gcc compiler. The bold numbers indicate where the -O2 flag outperformed the -O3 flag, thus the -O2 performance number is reported. These results lead us to expect the algorithm will exceed performance of 10000 hops per second in hardware implementations even on the smaller architectures usually found in constrained devices.

In some cases we anticipate a hub and spoke architecture where the hub tracks *all* user frequencies at every hop. In other words, the hub will maintain a sorted list of $(\text{USER\_ID}, E(\text{USER\_ID}))$ pairs at each time t. We assume this device has more computing capacity than the individual user devices.

The hub must encrypt each USER_ID and sort the list of pairs $(\text{USER\_ID}, E(\text{USER\_ID}))$ on the encrypted USER_ID value. In a bitsliced implementation, one solution is to compute each user's encrypted USER_ID the same way that individual users do, and then compute each user's index using the USER_j index computation algorithm 6.1. This tells us exactly where each user is in the sorted list of users, allowing the hub to insert each user into the sorted list at the appropriate index. This technique eliminates the need for a sorting algorithm and takes advantage of code that is already written for the users (slight modifications will be needed to save the user's encrypted USER_ID and return a $(\text{USER\_ID}, E(\text{USER\_ID}))$ pair).

7. Test Vector

Here is a test vector for Caju where the programmer sets the 128-bit Caju key rather than generating the key using SPECK. The test vector is for a state containing one user. The test vectors can be tested in the bitsliced implementation by initializing all users in the state to the initial state value. All user values should equal the final state value after the Caju algorithm.

```
Key (key[3] key[2] key[1] key[0]) = 0x9abcdef0 0x12345678 0x89abcdef 0x01234567
Initial State (W X Y Z) = 0x78 0x56 0x34 0x12
Final State (W X Y Z) = 0xcb 0x2a 0x62 0x93
```

	W	X	Y	Z	Round Key
Preadd :	0x79	0x75	0x71	0x75	0x01234567
Round 0:	0x89	0xde	0xde	0x84	0x89abcdef
Round 1:	0x4c	0xe6	0x88	0xf1	0x12345678
Round 2:	0x1e	0x81	0xa3	0x6d	0xdef89ab4
Round 3:	0xf9	0xda	0xa6	0x0d	0x54345474
Round 4:	0x72	0xd6	0x3c	0xe1	0xfdf9fdf1
Round 5:	0xc6	0x0d	0x67	0x90	0xbac89aaa
Round 6:	0x63	0x32	0xb5	0x59	0xc6e180ad
Round 7:	0x66	0x5f	0xf7	0x43	0x9317900e
Round 8:	0x02	0x78	0x68	0xe2	0x970a8613
Round 9:	0x93	0xb9	0x5b	0x77	0xf933b5b7
Round 10:	0xaf	0x63	0x56	0x50	0xe51e304b
Round 11:	0xcb	0x2a	0x62	0x93	0x993e67d4

Table 7.1: Test Vector for Caju

A. Bitsliced Implementation Details

This appendix describes an implementation of the PEAK SURFER frequency hopper using a bitsliced implementation of the CAJU block cipher.

SPECK128/256 is used in counter mode to generate the 128-bit keys used in CAJU. The details of SPECK can be found in [1].

A.1. Initialization

Refer to Table 6.1 for a glossary of variables that will be used throughout this exposition.

The state S is broken up into an SSIZE-long array S where each $S[i]$ consists of 4 words W, X, Y, and Z. Each word is stored in 8 registers, one corresponding to each bit level of that word (Z represents bit levels 0-7 of the overall 32-bit input, Y represents bit levels 8-15, X represents bit levels 16-23, and W represents bit levels 24-31).

The state should be initialized so that user j is initialized to their USER_ID j. Another way to think about this is to initialize each bit level i to 2^i 0's followed by 2^i 1's repeating until the entire bit level is filled. For example, bit level 0 will be repeating `0xa`'s, bit level 1 will be repeating `0xc`'s, etc.

The key structure used in this implementation is an array of four 32-bit values, referred to as k_0, k_1, k_2, and k_3. The 128-bit CAJU key will be initialized using the ciphertext output from SPECK128/256.

The SPECK128/256 plaintext value is initialized to the value `IV`. The key will be initialized to the preplaced TRANSEC cryptovariable `CV`.

A.2. Caju

For each hop, a new 128-bit key will be generated using SPECK128/256 and one application of Caju will occur per user, providing each user with their new index. The components of the Caju permutation that are user dependent can leverage a bitsliced implementation to increase efficiency. The user-independent SPECK128/256 and Caju key schedule computations were implemented in the standard (not bitsliced) way.

The Caju round function consists of three main components, the s-box function, the linear function, and the key add function.

A.2.1. Caju s-box Function

The s-box in Caju takes four bits as input and produces four bits of output. For a bitsliced implementation, the important observation is that this operation has a simple expression in terms of boolean operations, which means we can perform it more efficiently by storing the bits of multiple plaintexts in the bit levels of an array of machine words (same number of operations with more bits of output).

Algorithm A.1 Caju s-box Application

Input: State S

Output: State S updated in place

```
for i=0 to SSIZE do
  sbox(S[i]) //sbox function defined in Algorithm A.2
end for
```

Algorithm A.2 `sbox`: Apply Caju s-box to S_i

Input: State Subspace S[i] consisting of Words W, X, Y, Z

Output: State S updated in place

```
toffoli(X,W,Y) //toffoli function defined in Algorithm A.3
toffoli(Z,X,Y)
toffoli(Y,W,Z)
toffoli(W,X,Z)
```

Algorithm A.3 `toffoli`: Compute Toffoli Gate on three input Words

Input: Words X,Y,Z

Output: Word X updated in place

for i=0 **to** 7 **do**

`X.bit[i] = (X.bit[i]` $\oplus$ `(Y.bit[i] & Z.bit[i]))`

end for

A.2.2. Caju Linear Function

The Caju linear function consists of four left circular shifts and four `XOR`s. Left circular shifts are are efficient in bitsliced implementations as they involve simply modifying the bit levels used in the computation.

Algorithm A.4 Caju Linear Function

Input: State S

Output: State updated in place

for i=0 **to** SSIZE **do**

```
A_LCS_sigma_xor_B(S[i].W,S[i].X,2)
A_LCS_sigma_xor_B(S[i].X,S[i].W,6)
A_LCS_sigma_xor_B(S[i].W,S[i].X,1)
A_LCS_sigma_xor_B(S[i].X,S[i].W,7)

W_temp = S[i].W, X_temp = S[i].X

S[i].W = S[i].Y, S[i].Y = W_temp
S[i].X = S[i].Z, S[i].Z = X_temp
```

end for

A.2.3. Caju Key Add Function

The key add function adds a 32-bit key to a user's subcipher value. Since this function is implemented in a bitsliced manner, we can take advantage of the structure and `XOR` the key to the entire state. This involves picking off each bit of key, expanding the bit into a REGISTER_SIZE mask of all 0's or all 1's, and `XOR`ing the mask to the appropriate bit level of the state.

Algorithm A.5 `A_LCS_sigma_xor_B(S[i].A,S[i].B,sigma)`

Input: Word A

Input: Word B

Input: Integer shift value `sigma`

Output: Word A updated in place

```
A_copy = A
for i=0 to 7 do
  A.bit[i] = A_copy.bit[(i+sigma)%8] ⊕ B.bit[i]
end for
```

Algorithm A.6 `XOR` Round Key to the State

Input: State S

Input: 32-bit round key `key`

Output: State updated in place

```
for i=0 to SSIZE do
  keyadd2word(S[i].W,((key >> 24) & 0xFF))
  //keyadd2word function defined in Algorithm A.7
  keyadd2word(S[i].X,((key >> 16) & 0xFF))
  keyadd2word(S[i].Y,((key >> 8) & 0xFF))
  keyadd2word(S[i].Z,(key & 0xFF))
end for
```

Algorithm A.7 `XOR` Key to a Word

Input: Word W

Input: 8-bit portion of round key `k`

Output: Word updated in place

```
for i=0 to 7 do
  W.bit[i] ⊕= ((-1) * (k >> i) & 0x01)
end for
```

References

[1] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers. The SIMON and SPECK Families of Lightweight Block Ciphers. *IACR ePrint archive*, (404), 2013.